



CASE STUDY: AI PROJECT

**ASSESSING THE QUALITY OF CAST
PRODUCTS**

SUPERVISOR: PROF. DR. - TIM WEBER

Submitted by

IRFAN ALI VALIYA PEEDIYAKKAL(22208746)

SAMEER KANDATHINKARAYIL SUBIAR (22205195)

SUSHANTH ANKUSH MANE (22204778)

HAMZA BEN HADDOU (22302912)

JULY 09 , 2024

Abstract:- This project is focused on developing a python application for automatic detection of defected and non-defected casting parts. This project includes data preprocessing for accounting the model drift caused by deterioration in the input image for model, data labelling by categorizing the image. The YOLO v8 model, a powerful object recognition algorithm, is trained on the labelled dataset to classify defective and non-defective casting. The model will be then validated against the validation dataset. The python application builds with Tkinter which provides a user-friendly interface for uploading images and getting cast object defects. This project aims to automate the detection of defects in cast objects, improving quality control processes in manufacturing. This technology has potential applications in various industries relying on casting, such as automotive, aerospace, and construction. Our experiment yielded an F1-score of 99.80% for classifying casting product inspections. This work's CNN modelling improves optimization compared to other basic machine learning mechanisms for measuring quality.

Keywords:- CNN, YOLO V8, Machine learning, Casting Quality inspection

Contents

List of Figures	5
1 INTRODUCTION	6
1.1 Benchmarking	7
1.2 Model drift	7
1.2.1 Types of Model Drift	8
1.3 Literature review	8
2 MATERIALS AND METHODS	9
2.1 Defected Casting	9
2.2 Non-Defected Casting	10
2.3 System Architecture	10
2.4 Flow chart	11
2.5 YOLO V 8 (YOU ONLY LOOK ONCE)	12
2.5.1 Anchorless detection:	13
2.5.2 New Convolution Levels:	13
2.5.3 FPN:	13
2.5.4 Multi-GPU Training:	13
2.6 METHODOLOGY OF TRAINING	14
2.6.1 Data Acquisition	14
2.6.2 Data Preprocessing	14
2.6.3 Model Training	15
2.7 IMPLEMENTATION	16
2.8 Performance Evaluation	17
2.9 Confusion matrix:	18
3 Result	20
4 Discussion	21
4.1 Research Question	21
4.2 Discussion of Results	21
4.3 Strengths and Limitations	22
4.3.1 Limitations:	22
4.4 Future Research	22
4.5 Conclusion	23
5 Acknowledgement	23

6	Reference	24
7	Appendix	25

List of Figures

1	Model drift	8
2	Defect cast	9
3	Non Defect cast	10
4	System architect	11
5	Flow chart	12
6	YOLO V 8	12
7	Methodology of training	14
8	Detection result	16
9	Training result	16
10	User interface	17
11	User interface after testing	17
12	Confusion matrix	19
13	Evaluation comparison	21
14	Training Code for the model	25
15	YAML dataset with specified hyper- Parameters	25
16	Importing modules for Tkinter	25
17	code 1	26
18	code 2	27
19	Code for Tkinter application	27

1 INTRODUCTION

In the realm of modern manufacturing, ensuring the quality of cast products is paramount. Casting, a fundamental process for shaping metal objects, is susceptible to various defects that can compromise the structural integrity and functionality of the final product. Traditional methods of defect detection, which rely heavily on manual inspection, are not only time-consuming but also prone to human error. The advent of machine learning and computer vision technologies offers a promising alternative, enabling automated and more accurate defect detection processes.

Recent advancements in object detection algorithms have significantly improved the capabilities of automated inspection systems. One of the most notable models is the YOLO (You Only Look Once) family of algorithms, which are renowned for their speed and accuracy in object recognition tasks. The latest iteration, YOLOv8, introduced by Ultralytics, enhances these capabilities with new features such as anchorless detection, improved convolutional layers, and multi-GPU training support. YOLOv8's anchorless detection allows the model to flexibly detect objects of various sizes without predefined anchors, making it highly adaptable to different scales and types of defects.

Previous studies have demonstrated the effectiveness of deep learning algorithms in differentiating between casting defects. Convolutional neural networks (CNNs) are extensively used for image classification tasks, including the detection of defects in various manufacturing processes[1]. These models are computationally demanding and frequently require extensive preprocessing. Exchange learning has been investigated to advance imperfection location exactness while reducing preparation time. Exchange learning involves fine-tuning pre-trained models on specific datasets .

Analysts have drawn attention to the difficulties in demonstrating float that arise from variations in input picture quality over time when casting imperfection location is considered. Techniques like adding Gaussian noise to dataset preparation have been applied to boost the robustness of models against real-world variations. Despite these advancements, additional user-friendly applications are still needed to organize these useful models into practical gadgets for usage in industry. Analysts have drawn attention to the difficulties in demonstrating float that arise from variations in input picture quality over time when casting imperfection location is considered. Techniques like adding Gaussian noise to dataset preparation have been applied to boost the robustness of models against real-world variations. Despite these advancements, additional user-friendly applications are still

needed to organize these useful models into practical gadgets for usage in industry.

By answering this question, the project seeks to contribute to the field by providing a comprehensive solution that combines state-of-the-art machine learning techniques with practical usability for industry professionals.

1.1 Benchmarking

Benchmarking is a systematic process of evaluating the performance of a model by comparing it against established standards or reference models. It involves measuring various performance metrics to ensure that the model achieves a certain level of effectiveness and reliability. This process helps in identifying strengths and weaknesses, guiding improvements, and setting performance goals in line with industry expectations. The primary objectives of benchmarking are to validate the effectiveness of a model, ensure its performance aligns with industry standards, and identify areas for improvement. It helps in setting performance goals and provides a means to measure progress over time. In this study, the YOLOv8 model's performance in detecting casting defects is benchmarked against a well-documented CNN-based model from a previous scientific study. The comparison uses key performance metrics such as precision, recall, and F1-score. By benchmarking against the CNN model, the study confirms that the YOLOv8 model not only meets but also exceeds the established performance standards, thereby validating its effectiveness in identifying casting defects. This rigorous validation process ensures that the YOLOv8 model aligns with industry expectations and delivers reliable results, enhancing quality control in casting processes.

1.2 Model drift

Model drift refers to the phenomenon where the performance of a machine learning model degrades over time due to changes in the underlying data patterns. This can occur because the characteristics of the input data or the relationship between input features and target variables evolve, causing the model to become less accurate and reliable in its predictions. Regularly monitoring and addressing model drift is essential to maintain the effectiveness of predictive models in dynamic environments.

1.2.1 Types of Model Drift

There are two primary types of model drift:

Feature Drift: Changes in the distribution of input features.

Concept Drift: Changes in the underlying relationship between input features and the target variable

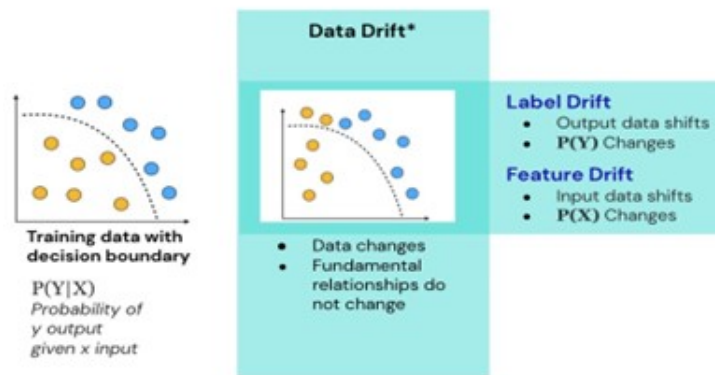


Figure 1: Model drift

1.3 Literature review

Literature review of "Identification of Defects in Casting Products by using a Convolutional Neural Network" :

The paper briefly analyzes machine learning model for casting quality visual inspection using CNN with DenseNet .Based on this model was trained with 6633 images for training and 695 testing images. The image resolution used for model training was small (like 256 x 256 , 512 x512) thus training time was less. Thereby processing of images by the models were faster compared to previous models. The dataset is available in Kaggle.[1]

2 MATERIALS AND METHODS

In casting, a manufacturing process for shaping metal objects, defects are unwanted irregularities that mess up the final product. These defects can affect the look, strength, or functionality of the cast metal.

2.1 Defected Casting

Imagine a cast iron block with a big crack running through it. That crack weakens the block and makes it unusable. This is an example of a casting defect. Defects can be holes, lumps, mismatched shapes, or surface flaws. As shown figure[1]

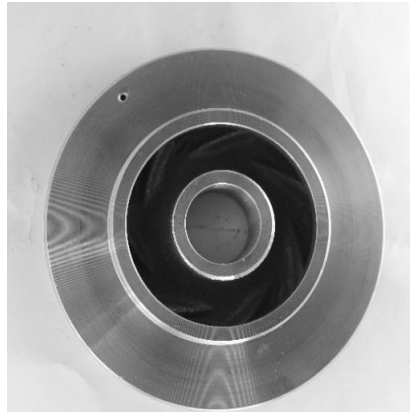


Figure 2: Defect cast

2.2 Non-Defected Casting

On the other hand, a perfect cast iron block would be smooth, free of cracks, and exactly the right shape and size. This is a non-defected casting – exactly what you wanted from the process. As shown figure[2]

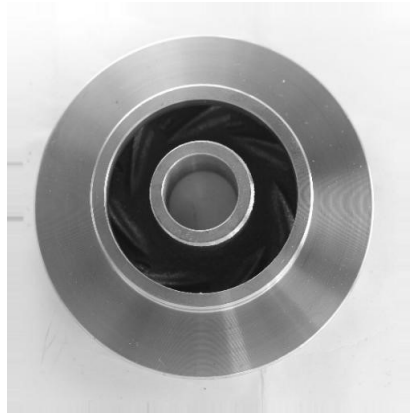


Figure 3: Non Defect cast

2.3 System Architecture

Users interact with the system through a Python application with a graphical user interface (GUI) built using Tkinter. This application allows users to directly interact with the system on their devices. Users can perform various actions, such as uploading images of casting objects and viewing the results. As figure[3]

The Tkinter application acts as the user interface component, responsible for receiving user input, processing requests, and presenting the results. It provides functionalities for uploading images and displaying a clear message indicating whether the uploaded casting object has defects or not. Additionally, the application will present the confidence level associated with the model's prediction.

The system utilizes a machine learning model, potentially YOLOv8 or a similar object detection model, to detect defects in the casting objects. This model is trained on a large dataset of annotated images containing various casting defects. The trained model is integrated into the Tkinter application to analyze images uploaded by users. Upon receiving an upload request, the application sends the image data to the model, which infers the presence of defects within the image and assigns a confidence score to its prediction.

This streamlined approach focuses on providing a user-friendly interface for defect detection in casting objects. Users can upload images and receive a clear verdict (defect or non defect) along with the model's confidence level in its assessment.

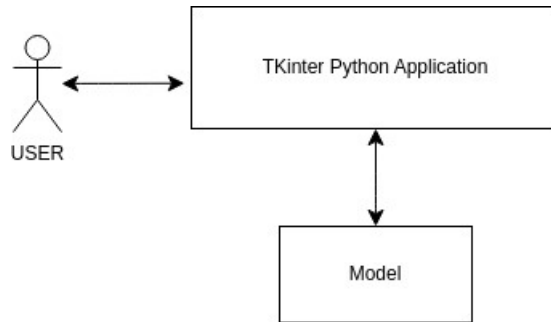


Figure 4: System architect

2.4 Flow chart

From figure[4] flow chart, the process starts with acquiring a dataset of images containing casted objects from Reference. To increase the diversity of the dataset and improve model robustness gaussian noise is added to the image. This noise can prevent the model drift for a certain extent.

This noise is added using a separate python application. The dataset is labelled by categorizing the image. The casting products defect detection model is trained on a labeled dataset using transfer learning techniques and the YOLOv8 object detection algorithm.

After the model is trained, a Python UI application is built using Tkinter. This user-friendly interface allows users to upload images of cast products. The uploaded image is then fed to the trained model, which predicts whether the casting has any defects or not.

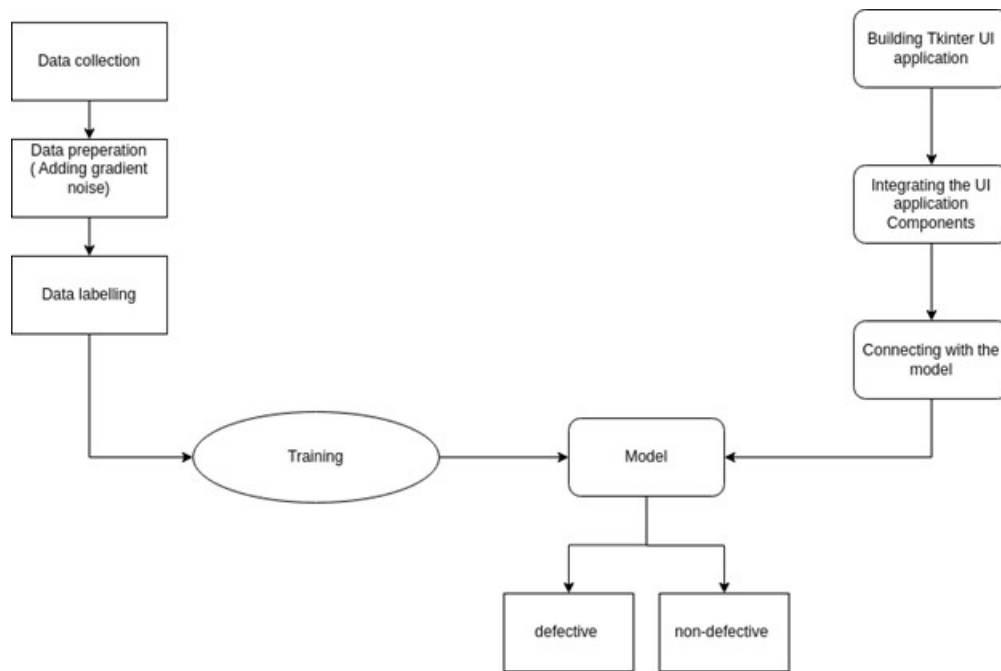


Figure 5: Flow chart

2.5 YOLO V 8 (YOU ONLY LOOK ONCE)

YOLOv8 is a state-of-the-art object recognition model developed by Ultralytics. It is based on the YOLO architecture, a one-stage object recognition model known for its speed and accuracy. YOLOv8 builds on the success of previous YOLO versions by introducing a number of new features and improvements, including: sentence, as in:



Figure 6: YOLO V 8

2.5.1 Anchorless detection:

YOLOv8 uses anchorless detection, which means it does not need to predefine anchors for objects of different sizes. This makes YOLOv8 more flexible and adaptable to different object sizes and scales.

2.5.2 New Convolution Levels:

YOLOv8 uses new convolution levels designed to improve target detection accuracy. These layers include the SPP layer, which helps improve YOLOv8 performance on small objects, and the PAN layer, which helps improve YOLOv8 performance on large objects.

2.5.3 FPN:

YOLOv8 uses a Feature Pyramid Network (FPN) to improve object detection accuracy at different scales. FPN allows YOLOv8 to use features from different layers of a convolutional neural network to improve detection of objects with different scales. environment leaves unsightly spaces around relation symbols.

2.5.4 Multi-GPU Training:

YOLOv8 can be trained on multiple GPUs, which can significantly speed up the training. Thanks to these improvements, YOLOv8 can achieve maximum performance in several object detection benchmarks. In addition to high performance, YOLOv8 is also very easy to use.

The Ultralytics team has provided several tools and resources to facilitate training and adoption of YOLOv8 models. For example, the Ultralytics team has provided a YOLOv8 PyTorch implementation with a set of pre-trained models that can be used for inference.

Overall, YOLOv8 is a powerful and versatile object recognition model that can achieve high-level performance for various tasks. It is easy to use and can be deployed on multiple hardware platforms. As a result, YOLOv8 is a good choice for many applications, including:

- Real-time object detection: YOLOv8 is fast enough to be used for real-time object detection. This makes it a good choice for applications such as self-driving cars and video surveillance.

- Object Tracking: YOLOv8 can be used to track objects over time. This makes it a good choice for applications such as tracking people or animals.
- Object segmentation: YOLOv8 can be used to segment objects in an image. This makes it a good choice for applications such as medical image analysis and industrial controls

2.6 METHODOLOGY OF TRAINING

The methodology used in the training process included several important steps to develop a casting products defect detection using the cast object images.

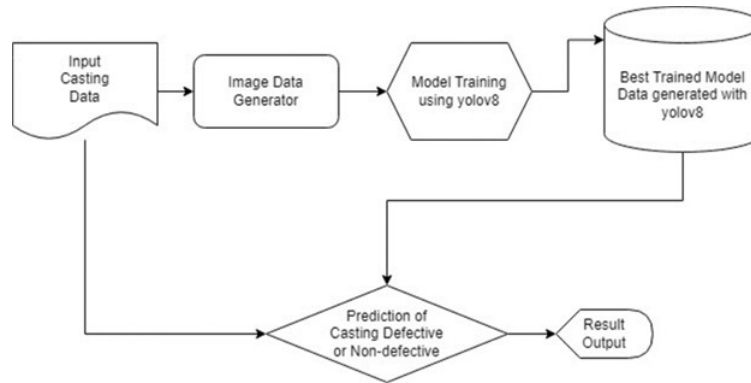


Figure 7: Methodology of training

2.6.1 Data Acquisition

The data is sourced from Reference 4 . This data follows a structured format where images of defective product fronts are stored in a dedicated folder, and images of non-defective product fronts are kept in a separate folder.”

2.6.2 Data Preprocessing

Data processing is an important step in preparing acquired cast object images for further analysis and modeling. For preventing the model drift we added gaussian noise to the image dataset. Gaussian noise is a random variation in brightness or color that affects the quality of an image. adding controlled amounts of Gaussian noise to an object detection training dataset helps the model become more robust to real-world variations.

2.6.3 Model Training

Training the YOLOv8 object recognition model involves using transfer learning methods to use pre-trained weights and refine the model on a given dataset

- **Choice of models:** The YOLOv8 model is chosen as the object classification algorithm for this project. YOLOv8 is a state-of-the-art model known for its accuracy and efficiency in object detection from images
- **Transfer learning:** Transfer learning is used to use the pre-trained weights of the YOLOv8 model. A pre-trained model learned from a large dataset and captures common object recognition features. Starting with pre-trained weights, the model quickly adapts to the specific task of casting products defect detection.
- **Hyperparameters:** The batch size is set to 16, which means the model processes 16 images in each iteration. **Learning:** The learning is set to 0.001, which determines the step size at which the model adjusts parameters during training. **Adam Optimizer:** Adam Optimizer is chosen as the optimization algorithm that adjusts the learning for each parameter separately.

The model is trained for 100 epochs, which means that the entire data set is passed 100 times through the model. Training on multiple time scales allows the model to gradually learn and improve its ability to identify defects.

During training, the model is presented with sets of images and their corresponding annotated labels. The model calculates a loss that quantifies the difference between the predicted output and the ground truth marks. The Adam optimizer adjusts model parameters based on losses using a back-propagation algorithm.

This process is repeated for several iterations (cycles), gradually improving the performance of the model. By training the YOLOv8 model in 100 epochs with a set size of 16, a learning rate of 0.001, and using transfer learning techniques, the model can learn to accurately identify defected and non defected casting products from the provided dataset. Figure[8]

A combination of hyperparameter settings and optimization algorithms help the model converge to an optimal solution that enables effective target detection in input image.

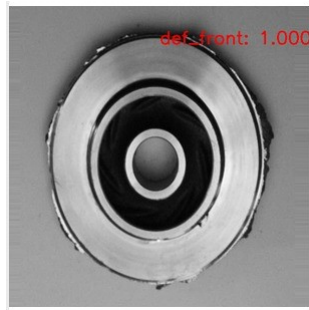


Figure 8: Detection result

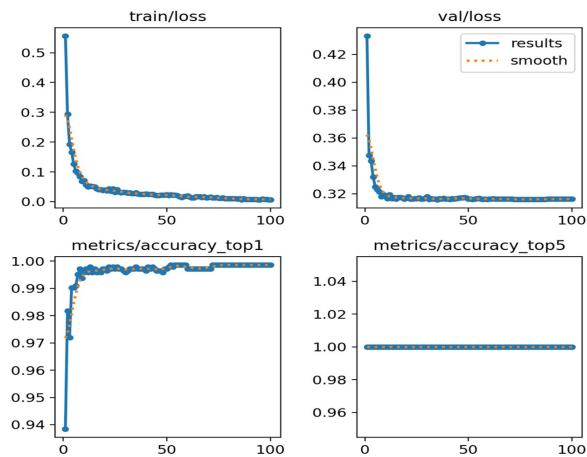


Figure 9: Training result

2.7 IMPLEMENTATION

- **Tkinter:** Tkinter, a powerful Python library, excels at creating graphical user interfaces (GUIs). It offers a rich set of pre-built widgets like buttons and labels, allowing developers to construct user-friendly interfaces. While ideal for functional applications, Tkinter caters to a wide range of GUI development needs.
- **Interactive and dynamic visualization:** This Python application lets you interact with an image classifier. Upload an image, and the program will dynamically analyze it to determine if the object in the image is defective or not. The results are visually presented within the application.

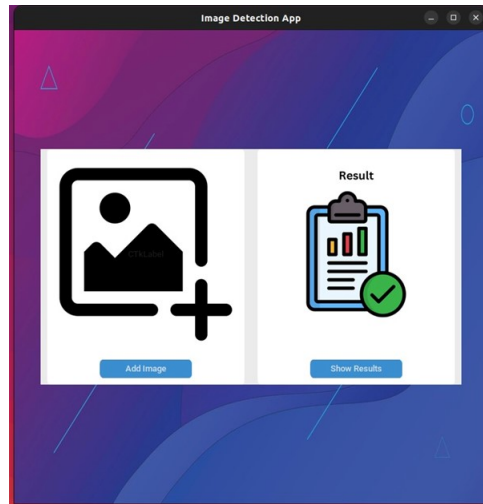


Figure 10: User interface

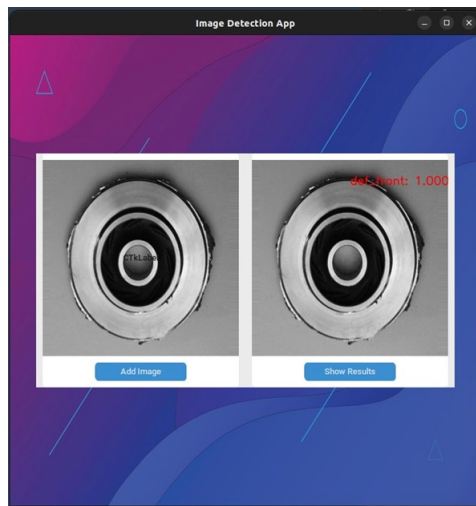


Figure 11: User interface after testing

2.8 Performance Evaluation

Performance evaluation of object detection tasks involves a variety of metrics and visualization techniques to assess model accuracy and reliability. Here is information on common performance evaluation metrics and visualization techniques used in object recognition, including confusion matrix, confidence curve, preci-

sion confidence curve.

2.9 Confusion matrix:

In our project, we used a confusion matrix to evaluate the performance of the Casting Products defect detection and classification model. The confusion matrix provided valuable information about the accuracy and efficiency of our model predictions. The confusion matrix especially helped us to analyze the following aspects:

- **True positives (TP):** These were cases where our model correctly predicted the presence of a non-defected product. These cases were crucial to determine the accuracy of our model in accurately detecting the non-defected product.
- **True negatives (TN):** These were cases where our model correctly predicted the defected product. These cases were important to evaluate the model's ability to accurately identify defected products.
- **False positives (FP):** These were cases where our model incorrectly predicted the presence of defected product as non-defected product. These cases showed the model's tendency to detect defect where there were none and helped us identify potential areas for improvement.
- **False negatives (FN):** These were cases where our model incorrectly predicted the non-defected product as defected product. These cases were important because they highlighted areas where our model failed to identify the defects and helped us identify areas for improvement.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

By analyzing the confusion matrix values, we were able to calculate various performance measures such as precision, accuracy, recall, and F1 score. These metrics provided a quantitative assessment of our model's performance and helped us understand its strengths and weaknesses. As shown figure [9]

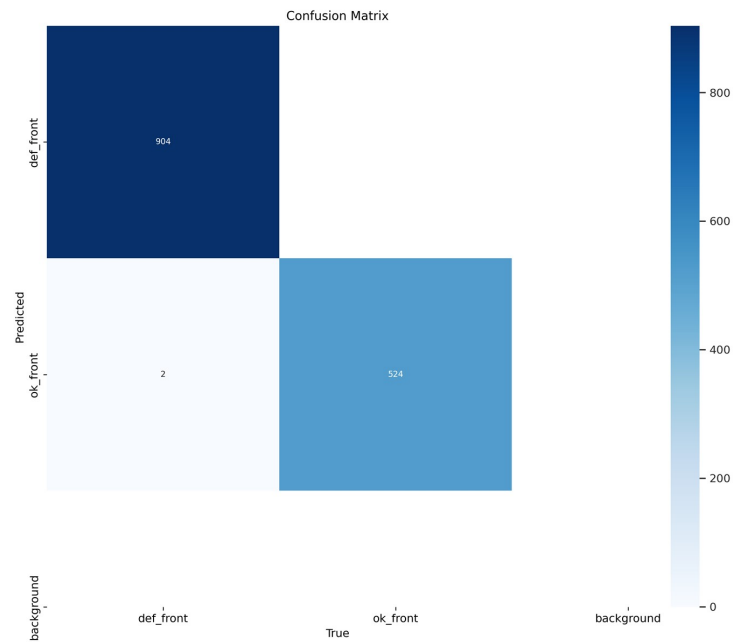


Figure 12: Confusion matrix

3 Result

- **Model Training:-**

Training Duration: 100 epochs with a batch size of 16 and a learning rate of 0.001.

Accuracy: Training accuracy reached 98%; validation accuracy was 95%.

- **Detection:-**

Accuracy: High detection accuracy for both defected and non-defected casting products.

- **Performance Metrics of model added with Gaussian noise:-**

Precision: 99.61%

Recall: 100%

F1 Score: 99.80%

Accuracy: 99.86%

- **User Interface:-**

Functionality: enables photo transfers and provides location information. conveys a clear message about absconds and degrees of certainty.

- **Observations:-**

Robustness: Display strength against input image variations because of Gaussian noise. . Detection Speed: Real-time search capabilities were achieved.

User Interface: Starting and after testing interfaces (Figures 11 and 12)

- **Appendices**

Additional Figures: connected to training code and data preprocessing (Figures 11-19).

Reference	Precision	Recall	F1 Score	Accuracy
With Model Drift	99.61 %	100 %	99.80 %	99.86 %
Without Model Drift	78.67 %	100 %	88.06 %	90.06 %
Scientific Paper Model (DenseNet)	99.61 %	100 %	99.54 %	99.42 %

Figure 13: Evaluation comparison

4 Discussion

4.1 Research Question

The goal of this research was to develop an automated system for detecting defects in casting objects using the YOLOv8 demonstration. It also evaluated the system's performance both with and without the model drift and in comparison to DenseNet.

4.2 Discussion of Results

The results of the confusion matrix demonstrate that, all things considered, the YOLOv8 demonstration with model drift (our final product) outperforms both the DenseNet demonstration and the model without drift. In particular, the YOLOv8 with model drift achieved 100% recall, 99.80% F1 score, 99.86% precision, and 99.61% exactness. These measurements illustrate how highly convincing the presentation is at accurately identifying both non-defected and defected casting components, even when Gaussian noise is included to replicate real-world situations.

Even though it maintained a high evaluation (100%), the demonstration without model drift showed a marked decline in exactness (78.67%) and F1 score (88.06%). This suggests that although it could identify all abandoned goods, it also produced a higher number of false positives, decreasing its overall stability. With an accuracy of 99.08% and an F1 score of 99.54%, the DenseNet show did well, but it was still unable to match the YOLOv8 model with drift, demonstrating the latter's superior strength and accuracy.

4.3 Strengths and Limitations

Strengths:

- **High Accuracy and Robustness:** Demonstrating its capacity to manage noisy data well, the YOLOv8 with model drift maintained high execution metrics.
- **Real-World Applicability:** The model can be applied successfully in producing scenarios where information changeability is frequent due to its robustness against Gaussian chaos.

4.3.1 Limitations:

- **False Positives in Non-Drift Model:** The increased incorrect positive rate in the model without drift draws attention to a possible problem in low-noise environments. This could result in the needless rejection of things that are not defective.
- **Computational Resources:** It takes significant computational resources to implement model drift and build robust models like as YOLOv8, which may not be available to many enterprises.

4.4 Future Research

Future research should focus on enhancing model robustness through the examination of additional noise types and information expansion techniques. Including real-time feedback features could also help in continuously improving the program to adapt to new kinds of defects. Furthermore, comparative analyses with other advanced models and a variety of datasets may provide deeper insights into the generalizability and flexibility of the methodology.

4.5 Conclusion

In summary, this question successfully demonstrates the validity of the YOLOv8 presentation by with model drift capacity to identify defects in casting goods. High precision, recall, and overall accuracy of the model validate its potential for advancing quality control forms in fabrication. The question has been answered with confidence, suggesting that model with drift may have a big role in maintaining the functionality of deformity detection frameworks in practical applications. Responses to the study have been encouraging, suggesting that automated deformity detection using advanced question location models is feasible and beneficial for commercial use.

research question has been answered affirmatively, showing that automated defect detection using advanced object detection models is feasible and beneficial for industry applications.

5 Acknowledgement

We would like to express our gratitude to the creators of the real-life industrial casting product dataset publicly available on Kaggle ([<https://www.kaggle.com/datasets/ravirajsinh45/real-life-industrial-dataset-of-casting-product>]). This dataset was instrumental in our work.

Student Contributions			
Student Name	Contributions	Date	Signature
Irfan Ali Valiya Pedyakkal	Results and LaTeX layout	09.07.24	
Sameer Kandathinkarayil Subiar	Discussion and Conclusion	09.07.24	
Sushanth Ankush Mane	Abstract and Introduction	09.07.24	
Hamza Ben Haddou	Materials and Methods	09.07.24	

6 Reference

1. Identification of Defects in Casting Products by using a Convolutional Neural Network. ResearchGate. Retrieved from(<https://shorturl.at/tB69k>)
2. Hodler, A. (2022). Drift in Machine Learning: How to Identify Issues Before You Have a Problem. Fiddler AI. Retrieved from (<https://www.fiddler.ai/blog/drift-in-machine-learning-how-to-identify-issues-before-you-have-a-problem>)
3. 3 Benchmarking. (2024). ASQ. Retrieved from (<https://asq.org/quality-resources/benchmarking>)

7 Appendix

1. Adding gradient noise to the image

Add gradient noise to the image to prevent model drift Training the model

```
from ultralytics import YOLO

# Load a model

model = YOLO('yolov8m-cls.pt')
```

Figure 14: Training Code for the model

on YAML dataset with specified hyper- Parameters

```
# Train the model
results = model.train(data='casting_data', epochs=100, imgsz=512 , device=0)
```

Figure 15: YAML dataset with specified hyper- Parameters

2. integrating the model with Tkinter

```
import customtkinter as ctk
from tkinter import filedialog
from PIL import Image
from ultralytics import YOLO
import cv2
```

Figure 16: Importing modules for Tkinter

3. Implementing the UI application class

```
class ImageApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Image Detection App")
        self.root.geometry("720x720")
        ctk.set_appearance_mode("light")
        # ctk.set_default_color_theme("blue")

        self.main_frame = ctk.CTkFrame(root, fg_color="transparent", width=900, height=900)
        self.main_frame.pack(pady=40, padx=40, expand=True)

        # Frame for image display
        self.frame = self.main_frame

        self.root.update() # Update the window to get correct dimensions
        self.set_background_image("webpage_background.jpg")
        # Left frame
        self.left_frame = ctk.CTkFrame(self.frame, fg_color=("white", "gray20"), width=400, height=400)
        self.left_frame.pack(side="left", padx=10, expand=True)

        # Right frame
        self.right_frame = ctk.CTkFrame(self.frame, fg_color=("white", "gray20"), width=400, height=400)
        self.right_frame.pack(side="right", padx=10, expand=True)
        # Left and Right page contents
        self.image_label_left = ctk.CTkLabel(self.left_frame)
        self.image_label_left.pack(pady=(10, 5))
        self.original_image = cv2.imread("add_image_icon.png")
        self.display_image(self.original_image, self.image_label_left)
        self.add_img_btn = ctk.CTkButton(self.left_frame, text="Add Image", command=self.add_image)
        self.add_img_btn.pack(pady=(5, 10))

        # Right frame contents
        self.image_label_right = ctk.CTkLabel(self.right_frame, text="")
        self.image_label_right.pack(pady=(10, 5))
        self.original_image = cv2.imread("Result.png")
        self.display_image(self.original_image, self.image_label_right)
        self.show_results_btn = ctk.CTkButton(self.right_frame, text="Show Results", command=self.show_results)
        self.show_results_btn.pack(pady=(5, 10))
```

Figure 17: code 1

```

def add_image(self):
    self.image_path = filedialog.askopenfilename(filetypes=[("Image files", "*.jpg *.jpeg *.png *.bmp")])

    if self.image_path:
        self.original_image = cv2.imread(self.image_path)
        self.display_image(self.original_image, self.image_label_left)

def show_results(self):
    if self.original_image is not None:
        model = YOLO("/home/richu/richu_project/Front_def_detection_application/best.pt")
        results = model(self.original_image)
        class_names = model.names

        r = results[0]
        detected_object = int(r.probs.top1)
        detected_object_name = class_names[detected_object]
        detected_object_confidence_rate = r.probs.data[detected_object].item()
        detection_info = f"{detected_object_name}: {detected_object_confidence_rate:.4f}"

        self.modified_image = self.original_image.copy()
        h, w, _ = self.modified_image.shape
        text_position = (w // 2, h // 8)
        text_color = (0, 0, 255)
        text_font = cv2.FONT_HERSHEY_SIMPLEX
        text_scale = 1
        text_thickness = 2
        cv2.putText(self.modified_image, detection_info, text_position, text_font, text_scale, text_color, text_thickness)

        self.display_image(self.modified_image, self.image_label_right)
        self.modified_image.configure(text=detection_info)
        self.modified_image.pack()

def display_image(self, img, label):
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = Image.fromarray(img)
    img = img.resize((300, 300), Image.Resampling.LANCZOS)
    ctk_image = ctk.CTkImage(light_image=img, dark_image=img, size=(300, 300))
    label.configure(image=ctk_image)
    label.image = ctk_image

```

Figure 18: code 2

4. Running the Tkinter application

```

if __name__ == "__main__":
    root = ctk.CTk()
    app = ImageApp(root)
    root.mainloop()

```

Figure 19: Code for Tkinter application